Modélisation de la diffusion des gliomes

Approche mathématique et numérique

Polytech Lyon · Maths Appliquées · 2024–2025

Ce document présente la démarche complète de modélisation mathématique et numérique de la propagation d'un glioblastome dans le cerveau. À partir des données IRM du challenge BraTS, nous simulons l'évolution spatio-temporelle de la densité tumorale grâce au modèle de réaction-diffusion de Swanson, discrétisé par un schéma de Crank-Nicolson sur grille 3D. Le document couvre les fondements mathématiques de l'EDP, la démarche technique d'implémentation en Python, et le pipeline général du projet de la biologie à la validation quantitative.

Paramètre	Valeur / Description
Modèle	Fisher-KPP / Swanson — EDP de réaction-diffusion
Données	BraTS 2021 — IRM multimodale NIfTI (240×240×155 voxels)
Langage	Python 3.10+ — nibabel, numpy, scipy.sparse, matplotlib, skimage
Équation	dc/dt = div(D(x) grad(c)) + rho * c * (1 - c)
Paramètres	D_w = 0.013 mm²/j (matière blanche), D_g = 0.0013 mm²/j (grise), rho ~ 0.018 j■¹
Schéma num.	Crank-Nicolson — stable, ordre 2 en temps — résolution creuse (spsolve)
Objectif final	Visualisation 3D de la propagation + validation Dice ≥ 0.65

Sommaire

Introduction — contexte et objectifs 3

Section 1 — Fondements mathématiques 4

1.1 L'équation de Fisher-KPP et le modèle de Swanson 4

1.2 La variable c(x,t) et son interprétation 4

1.3 Le terme de diffusion : D(x) anisotrope 5

1.4 Le terme de prolifération logistique 6

1.5 Modèle complet, conditions aux limites 6

1.6 Tableau de correspondance termes / biologie 7
1.7 Le ratio D/rho et le phénotype tumoral 7
Section 2 — Pipeline général du projet 8
2.1 Vue d'ensemble des 5 phases 8
2.2 Détail mathématique et numérique 8
Section 3 — Démarche d'implémentation Python 9
3.1 Librairies et organisation des fichiers 9
3.2 Les 6 modules : fonctions et en-têtes 10
3.3 Fichier principal main.py 13
3.4 Visualisation 3D — isosurface Marching Cubes 13
Références 14

Introduction — contexte et objectifs

Contexte médical

Les glioblastomes (GBM) sont les tumeurs cérébrales primaires les plus agressives chez l'adulte, avec une survie médiane d'environ 15 mois après diagnostic. Leur caractéristique principale est leur capacité d'invasion diffuse du tissu cérébral sain, bien au-delà des marges visibles à l'IRM. Les cellules cancéreuses migrent préférentiellement le long des faisceaux d'axones myélinisés de la matière blanche, rendant toute résection chirurgicale complète pratiquement impossible.

La modélisation mathématique de cette propagation permet de simuler et prédire l'évolution tumorale de façon patient-spécifique, d'estimer l'étendue de l'invasion invisible à l'imagerie, et d'optimiser les marges de traitement par radiothérapie.

Notre approche : le modèle de Swanson

Nous adoptons le modèle de réaction-diffusion de Swanson (2000–2003), qui décrit l'évolution de la densité cellulaire tumorale c(x,t) par une EDP de type Fisher-KPP. Ce modèle est largement utilisé dans la littérature de neuro-oncologie computationnelle, validé sur données cliniques, et suffisamment simple pour être implémentable avec des outils Python standards.

Équation centrale du projet : dc/dt = div(D(x) · grad(c)) + rho · c · (1 - c)

Objectifs du projet

Objectif	Description
Modélisation mathématique	Comprendre et formaliser l'EDP de Swanson — diffusion anisotrope D(x) et prolifération logistique rho·c·(1–c)
Discrétisation numérique	Implémenter le Laplacien 3D creux et le schéma Crank-Nicolson pour la résolution temporelle
Pipeline Python complet	Développer 6 modules (loader, preprocessing, diffusion_map, laplacian, simulation, visualization)
Données réelles BraTS	Travailler sur des IRM cérébraux réels du challenge BraTS 2021 au format NIfTI
Calibration des paramètres	Estimer D et rho par MCMC en comparant la simulation aux IRM longitudinaux
Visualisation 3D	Produire une isosurface 3D de la densité tumorale simulée via Marching Cubes
Validation quantitative	Mesurer la qualité des prédictions par des métriques standard (Dice ≥ 0.65, HD95 ≤ 8 mm)

Données : le challenge BraTS

Le Brain Tumor Segmentation Challenge (BraTS) fournit des IRM multimodaux de patients atteints de gliomes, avec segmentations expertes. Chaque patient dispose de quatre modalités IRM au format NIfTI (.nii.gz), recalées sur une grille isotrope de 240×240×155 voxels à résolution 1 mm :

Fichier	Modalité	Utilité dans notre modèle
t1.nii.gz	T1 natif	Anatomie générale — segmentation tissu WM/GM
t1ce.nii.gz	T1 + gadolinium	Tumeur active rehaussante — coeur nécrotique
t2.nii.gz	T2	Oedème péritumoral
flair.nii.gz	FLAIR	Envahissement diffus — zone d'invasion
seg.nii.gz	Segmentation	Vérité terrain : labels 0 (sain), 1, 2, 4 (tumeur)

Section 1 — Fondements mathématiques

1.1 L'équation de Fisher-KPP et le modèle de Swanson

Le modèle de Swanson (2000) repose sur une EDP de réaction-diffusion de type Fisher-KPP. Initialement proposé pour modéliser la propagation d'espèces biologiques, ce modèle a été adapté aux gliomes en distinguant la migration cellulaire (diffusion) de la division cellulaire (réaction) :

$$dc/dt = div(D(x) \cdot grad(c)) + rho \cdot c \cdot (1 - c)$$

Terme	Mécanisme	Rôle biologique
div(D(x) grad(c))	Migration cellulaire	Étalement spatial de la tumeur dans le tissu
rho · c · (1-c)	Division cellulaire	Croissance locale avec saturation

1.2 La variable c(x,t) et son interprétation

c est la densité locale normalisée de cellules tumorales : c(x,t) : R³ x R■ → [0, 1]. Elle représente la fraction volumique de glioblastome à chaque voxel (1 mm³). Ce n'est pas un comptage de cellules individuelles mais une description continue et macroscopique, valide car le nombre de cellules par voxel est très grand (~10■).

Valeur de c	Signification biologique
c = 0	Tissu sain, aucune cellule cancéreuse
0 < c < 1	Tissu partiellement envahi
c = 1	Tissu à capacité maximale (saturation locale)

La condition initiale c(x, 0) = c_0(x) est extraite de la segmentation IRM BraTS (masque de tumeur visible sur la séquence FLAIR ou T1-Gd).

1.3 Le terme de diffusion : D(x) anisotrope

Le terme div(D(x) grad(c)) décrit la migration des cellules tumorales dans le tissu. D(x) n'est pas un scalaire uniforme : c'est un tenseur 3×3 qui encode la vitesse et la direction de migration selon la microstructure locale du cerveau.

Valeurs du coefficient de diffusion par type de tissu :

Tissu	Notation	Valeur typique	Raison biologique
Matière grise (GM)	D_g	0.0013 mm²/j	Structure compacte, corps neuronaux — migration lente
Matière blanche (WM)	D_w	0.013 mm²/j	Axones myélinisés — migration rapide le long des fibres

LCR / bord du crâne	D_csf	0.0 mm²/j	Pas de migration tumorale dans le liquide

Le rapport D_w / D_g = 10 signifie que les cellules migrent 10 fois plus vite dans la matière blanche. C'est pourquoi les gliomes ont des formes irrégulières à l'IRM : ils suivent les faisceaux axonaux myélinisés (corps calleux, capsule interne, etc.).

Forme développée du terme de diffusion :

Lorsque D varie spatialement — le cas réel — la forme complète est :

```
div(D · grad(c)) = D · Laplacien(c) + grad(D) · grad(c)
```

Le second terme grad(D) · grad(c) est crucial : il représente la dérive aux interfaces matière blanche/grise. Quand une cellule passe d'une zone à faible D vers une zone à fort D, elle accélère. C'est ce terme qui génère les extensions digitiformes (doigts tumoraux) caractéristiques des GBM observés à l'IRM FLAIR.

1.4 Le terme de prolifération logistique rho·c·(1−c)

Ce terme issu du modèle de Verhulst (1838) décrit la croissance locale de la population tumorale avec une capacité limite. Contrairement à la croissance exponentielle rho·c (valable si l'espace est infini), le facteur (1−c) freine la prolifération à mesure que le tissu se sature :

Densité c	Facteur (1-c)	Interprétation
c ≈ 0	≈ 1	Tissu sain — prolifération maximale, beaucoup d'espace libre
c = 0.5	= 0.5	Tissu à moitié envahi — régime intermédiaire (pic de prolifération)
c → 1	→ 0	Tissu saturé — compétition pour l'espace, croissance nulle

Le pic de prolifération est atteint pour c = 1/2 avec un taux maximal de rho/4. La capacité de portage c = 1 est limitée biologiquement par l'oxygène, le glucose, et l'espace entre les neurones.

Source	Valeur de rho
Swanson et al. (2000)	rho ≈ 0.012 j■¹
Swanson et al. (2003)	rho ≈ 0.025 j■¹
Plage typique GBM	0.012 – 0.025 j■¹ → doublement en 28–58 jours

1.5 Modèle complet et conditions aux limites

Le modèle de Swanson complet, dans sa forme anisotrope 3D :

```
dc/dt = div( D(x) · grad(c) ) + rho · c · (1 - c) c(x, 0) = c_0(x) [condition initiale : segmentation IRM] dc/dn = 0 sur bord(Omega) [Neumann : pas de flux aux parois du crane]
```

La condition de Neumann homogène sur le bord du domaine cérébral signifie que les cellules ne traversent pas le crâne — biologiquement correct pour les gliomes. Le principe du maximum garantit que c reste dans [0, 1] pour tout t > 0 si c_0 ∈ [0, 1].

1.6 Tableau de correspondance termes / biologie

Terme	Signification math.	Biologie tumorale	Dans notre projet
c(x,t)	Densité normalisée ∈ [0,1]	Fraction volumique de cellules cancéreuses	Valeur par voxel BraTS
dc/dt	Dérivée temporelle locale	Vitesse d'invasion du tissu sain	Evolution entre IRM longitudinaux
D(x)	Tenseur 3×3 de diffusion	Motilité cellulaire selon le tissu	D_w=0.013, D_g=0.0013 mm²/j
div(D grad(c))	Divergence du flux diffusif	Migration le long des fibres axonales	Laplacien creux 3D scipy
rho	Scalaire > 0 (j■¹)	Taux net de division cellulaire	Calibré par MCMC, ≈ 0.018 j■¹
rho·c·(1-c)	Terme logistique de Verhulst	Croissance bornée par le tissu	Pic à c=0.5, nul à c=0 et c=1
D/rho	Ratio diffusion/prolifération	Phénotype tumoral dominant	Grand→invasif, petit→prolifératif

1.7 Le ratio D/rho et le phénotype tumoral

Le ratio D/rho (en mm²) est la grandeur diagnostique centrale du modèle. Il détermine la forme et le comportement de la tumeur simulée, et correspond à une réalité clinique observable à l'IRM :

D/rho grand (> 1 mm²)	D/rho petit (< 0.1 mm²)
Diffusion rapide, division lente	Diffusion lente, division rapide
Gliome très infiltrant, tentaculaire	Masse compacte et sphérique
Frontières floues à l'IRM FLAIR	Anneau rehaussant net au T1-Gd
Difficile à réséquer chirurgicalement	Plus accessible à la chirurgie

La longueur caractéristique d'invasion est définie par : l = sqrt(D/rho). Pour les valeurs typiques de GBM : l ≈ 0.85 mm (matière blanche) et 0.27 mm (grise). En pratique, l'envahissement invisible peut s'étendre à plusieurs centimètres dans la matière blanche — c'est précisément ce que notre simulation cherche à quantifier.

Section 2 — Pipeline général du projet

Le projet s'organise en cinq phases successives, de la collecte des données biologiques jusqu'à la validation quantitative des prédictions. Chaque phase produit des sorties qui alimentent la suivante.

2.1 Vue d'ensemble des 5 phases

Phase 0	**Données IRM (BraTS)**	Fichiers NIfTI — T1, FLAIR, segmentation
▼		
Phase 1	**Modèle mathématique**	EDP Fisher-KPP · D(x) anisotrope · ρ logistique
▼		
Phase 2	**Discrétisation numérique**	Différences finies · Laplacien creux 3D · Crank-Nicolson
▼		
Phase 3	**Implémentation Python**	6 modules · nibabel / numpy / scipy.sparse / skimage
▼		
Phase 4	**Calibration MCMC**	Estimation D et ρ par comparaison simulation ↔ IRM réel
▼		
Phase 5	**Validation quantitative**	Dice ≥ 0.65 · HD95 ≤ 8 mm · 6 métriques standard

2.2 Décomposition mathématique et numérique

La Phase 1 et la Phase 2 sont intimement liées : chaque terme mathématique de l'EDP correspond à un bloc numérique précis dans l'implémentation.

Terme mathématique	Représentation numérique	Fonction Python
div(D(x) grad(c))	Laplacien creux 3D (matrice scipy CSR)	build_laplacian_3d()
D(x) — carte de diffusion	Tableau numpy 3D de coefficients	build_diffusion_map()
grad(D)·grad(c) — terme interface	Inclus dans les coefficients du Laplacien	automatique
rho·c·(1-c) — prolifération	Calcul vectoriel numpy sur c aplati	proliferation_term()
Schéma Crank-Nicolson	Deux matrices creuses A et B (pré-calculées)	build_cn_matrices()

Système linéaire (I-dt/2·L)c^{n+1}=rhs	Résolution par spsolve à chaque pas	run_simulation()

Section 3 — Démarche d'implémentation Python

3.1 Librairies et organisation des fichiers

Librairie	Utilité	Complexité
nibabel	Lire les fichiers IRM NIfTI (.nii.gz)	Facile — 2 fonctions à connaître
numpy	Manipuler les grilles 3D de voxels	Familier
scipy.sparse	Stocker le Laplacien creux N×N	Moyen — évite 650 Go RAM en dense
scipy.sparse.linalg	Résoudre le système linéaire creux	Facile — une seule fonction spsolve
matplotlib	Visualisation 2D (coupes, courbes)	Familier
scikit-image	Isosurface 3D via Marching Cubes	Facile — une seule fonction

Organisation du dépôt Git :

```
projet-tumeur/
main.py ← point d'entree
src/
__init__.py
loader.py ← etape 1 : chargement NIfTI
preprocessing.py ← etape 2 : masques, condition initiale
diffusion_map.py ← etape 3 : carte D(x)
laplacian.py ← etape 4 : operateur Laplacien creux
simulation.py ← etape 5 : boucle Crank-Nicolson
visualization.py ← etape 6 : coupes 2D et isosurface 3D
data/BraTS2021_00001/ ← donnees patient (non versionnees)
docs/ ← ce document et les fiches
requirements.txt ← nibabel numpy scipy matplotlib scikit-image
```

3.2 Les 6 modules — fonctions et en-têtes

Module 1 : loader.py — chargement des données IRM

Lit les fichiers NIfTI et extrait le tableau 3D et la taille physique des voxels.

```
def load_nifti(filepath: str) -> tuple[np.ndarray, np.ndarray]:
# Retourne : (volume 3D, matrice affine 4x4)
# Utilise : nibabel.load(filepath).get_fdata()

def get_voxel_size(filepath: str) -> tuple[float, float, float]:
# Retourne : (dx, dy, dz) en mm — generalement (1.0, 1.0, 1.0) pour BraTS
# Utilise : nib.load(filepath).header.get_zooms()
```

Module 2 : preprocessing.py — prétraitement et masques tissulaires

Normalise les intensités IRM, extrait le masque cerveau, segmente les tissus par seuillage des intensités T1, et extrait la condition initiale depuis la segmentation BraTS.

```
def normalize_volume(volume: np.ndarray) -> np.ndarray:
# Normalise entre 0 et 1 par percentiles p1/p99

def extract_brain_mask(volume: np.ndarray, threshold: float = 0.01) -> np.ndarray:
# Masque binaire — voxels > threshold appartiennent au cerveau
# Optionnel : scipy.ndimage.binary_fill_holes pour boucher les trous

def build_tissue_masks(t1_norm, brain_mask) -> tuple[ndarray, ndarray, ndarray]:
# Retourne (wm_mask, gm_mask, csf_mask) par seuillage T1
# Seuils indicatifs : WM > 0.75, GM 0.40-0.75, CSF < 0.40

def get_initial_tumor(seg_volume: np.ndarray) -> np.ndarray:
# Extrait c_0 depuis les labels BraTS (1=necrose, 2=oedeme, 4=actif)
# Retourne tableau float dans {0.0, 1.0}
```

Module 3 : diffusion_map.py — carte de diffusion D(x)

Construit le tableau 3D qui associe à chaque voxel son coefficient D selon le type de tissu.

```
D_WHITE_MATTER = 0.013 # mm²/jour
D_GRAY_MATTER = 0.0013 # mm²/jour
D_CSF = 0.0 # pas de migration dans le liquide

def build_diffusion_map(wm_mask, gm_mask, csf_mask,
d_white=D_WHITE_MATTER, d_gray=D_GRAY_MATTER) -> np.ndarray:
# Retourne tableau 3D float — D_map[i,j,k] = D du voxel

def rescale_diffusion_for_voxel_size(D_map, voxel_size_mm) -> np.ndarray:
# Correction si voxels != 1mm : D_num = D_phys / h²
# Pour BraTS (h=1mm) : ne change rien
```

Module 4 : laplacian.py — opérateur Laplacien creux 3D

C'est le module le plus délicat. Il construit la matrice creuse N×N (N = voxels dans le masque) qui encode l'opérateur div(D grad(·)) par différences finies centrées. Les coefficients aux interfaces sont la moyenne des D voisins (schéma conservative).

```
Discretisation : [div(D grad(c))]_{i,j,k} = somme sur les 6 voisins de D_{i+1/2} *
(c_{voisin} - c_{i,j,k}) avec D_{i+1/2} = (D_i + D_voisin) / 2
```

```
    def flatten_index(i, j, k, shape) -> int:
    # Equivalent a np.ravel_multi_index((i,j,k), shape)

    def build_laplacian_3d(D_map: np.ndarray, brain_mask: np.ndarray) -> csr_matrix:
    # 1. Creer lil_matrix (facile a remplir ligne par ligne)
    # 2. Pour chaque voxel du masque : ajouter les 6 voisins
    # Condition Neumann : ignorer les voisins hors masque
    # 3. Convertir en csr_matrix pour la resolution
    # VALIDATION : somme de chaque ligne doit etre nulle (Laplacien discret)
```

Module 5 : simulation.py — boucle Crank-Nicolson

Le schéma de Crank-Nicolson est la moyenne de l'Euler explicite et implicite. Il est stable inconditionnellement pour la diffusion et d'ordre 2 en temps. À chaque pas de temps, il faut résoudre un système linéaire creux.

```
Schema : (I - dt/2 · L) · c^{n+1} = (I + dt/2 · L) · c^n + dt · F(c^n) ou F(c) = rho
· c · (1 - c) et L = matrice Laplacien
```

```
    def build_cn_matrices(L: csr_matrix, dt: float) -> tuple[csr_matrix, csr_matrix]:
    # Retourne (A, B) = (I - dt/2·L, I + dt/2·L) — calculees une seule fois

    def proliferation_term(c: np.ndarray, rho: float) -> np.ndarray:
    # Retourne rho * c * (1 - c) [vectoriel sur tableau 1D]
    # Clipper c dans [0,1] avant le calcul

    def run_simulation(c0, A, B, rho, dt, n_steps, brain_mask,
    save_every=10) -> list[np.ndarray]:
    # Boucle : rhs = B·c + dt·F(c) puis c_new = spsolve(A, rhs)
    # Retourne liste de snapshots 3D (volumes reconstruits)

    def unflatten_volume(c_flat, brain_mask) -> np.ndarray:
    # volume = np.zeros(brain_mask.shape)
    # volume[brain_mask] = c_flat [une seule ligne]
```

Module 6 : visualization.py — coupes 2D et isosurface 3D

Deux niveaux de visualisation : les coupes axiales (rapides, pour déboguer) et l'isosurface 3D par Marching Cubes (visuel final, nécessite scikit-image).

```
def plot_axial_slices(snapshots, slice_idx=None, times_days=None, cmap='hot'):
# Grille de coupes volume[:,:,z] a differents temps — plt.subplot()

def plot_3d_isosurface(volume, iso_level=0.15, brain_mask=None):
# from skimage.measure import marching_cubes
# verts, faces, _, _ = marching_cubes(volume, level=iso_level)
# Affichage via Poly3DCollection (mpl_toolkits.mplot3d)
# iso_level bas (0.05-0.15) = invasion diffuse
# iso_level haut (0.5-0.8) = coeur tumoral

def downsample_volume(volume, factor=2) -> np.ndarray:
# volume[::factor, ::factor, ::factor] pour accelerer la visu
```

3.3 Fichier principal main.py

Le fichier main.py orchestre l'ensemble du pipeline sans contenir de logique métier. Il définit les paramètres de simulation en haut du fichier et appelle les fonctions des modules dans l'ordre.

```
# Parametres de simulation
RHO = 0.018 # taux de proliferation (j■¹)
DT = 1.0 # pas de temps (jours)
N_STEPS = 300 # 300 jours de simulation
ISO_LEVEL= 0.15 # niveau isosurface pour la visu 3D

# Pipeline complet
t1, affine = load_nifti('t1.nii.gz')
t1_norm = normalize_volume(t1)
brain_mask = extract_brain_mask(t1_norm)
wm, gm, csf = build_tissue_masks(t1_norm, brain_mask)
c0 = get_initial_tumor(seg)
D_map = build_diffusion_map(wm, gm, csf)
L = build_laplacian_3d(D_map, brain_mask) # etape la plus lente
A, B = build_cn_matrices(L, DT)
snapshots = run_simulation(c0[brain_mask], A, B, RHO, DT, N_STEPS, brain_mask)
plot_3d_isosurface(snapshots[-1], iso_level=ISO_LEVEL, brain_mask=brain_mask)
```

3.4 Conseils pratiques pour démarrer

Tester sur un sous-volume — Travailler d'abord sur un cube 50x50x50 voxels centré sur la tumeur. Valider chaque module en secondes avant de passer au cerveau complet.

Ordre de développement — loader → preprocessing → diffusion_map → laplacian (le plus difficile) → simulation sur petit volume → volume complet → visualisation.

Valider le Laplacien — Sur un cube homogène (D constant), la somme de chaque ligne de la matrice doit être nulle. Test rapide et fiable avant de lancer la simulation.

Commencer avec D constant — Utiliser D_map = np.full(shape, 0.013) pour valider le schéma numérique isolément, avant d'activer la segmentation tissu.

Performance — La construction du Laplacien sur le cerveau complet peut prendre 5-15 min en Python pur. Tester sur sous-volume, puis lancer en arrière-plan.

Références

Auteur(s)	Source	Contribution
Swanson K.R. et al.	Cell Proliferation 33(5), 2000	Modele original D_w/D_g pour gliomes
Swanson K.R. et al.	Br. J. Cancer 86(3), 2003	Calibration patient-specifique sur IRM
Murray J.D.	Mathematical Biology II, Springer 2003	Fondements EDP reaction-diffusion
Harpold H.L. et al.	J. Neuropathol. 66(1), 2007	Revue complete du modele de Swanson
Konukoglu E. et al.	MICCAI 2007	Extension anisotrope avec tenseur DTI
BraTS Challenge	www.med.upenn.edu/cbica/brats	Donnees IRM et segmentations
scipy.sparse	docs.scipy.org/doc/scipy/reference/sparse.html	Documentation Laplacien creux
nibabel	nipy.org/nibabel/gettingstarted.html	Lecture NIfTI en Python
skimage.measure	scikit-image.org/docs/stable/api/skimage.measure.html	Marching Cubes 3D